Lecture 19:

Introduction to React

-By Instructor Name

K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Course - Web Development - II

welcome@krmangalam.edu.in

Sohna Road,Gurugram,Haryana 122103

Table of Contents

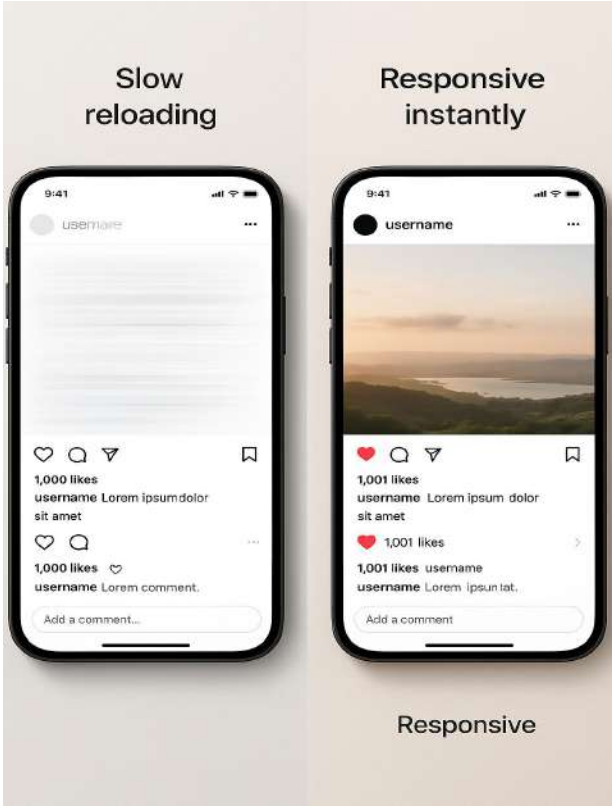
- React Overview
- React vs Angular vs Vue
- Why Choose react?
- References.
- The End.

What if your entire Instagram page reloaded every time you liked a photo?

What if your entire Instagram page reloaded every time you liked a photo?

That would be slow and frustrating — reloading the whole page just to like a post.

React fixes this by updating **only the part that changes**, like the like button, making apps feel fast and smooth — just like Instagram or Facebook.



What is React?

React is a JavaScript library for building user interfaces, developed by Facebook and released in 2013.

It is not a complete "framework" because it only handles the "view" layer of an application. You choose your own tools for routing, state management, etc.

React helps developers build **dynamic**, **responsive**, and **interactive** user interfaces without unnecessary reloads — and that's why it's one of the most popular JavaScript libraries in the world.

Imperative vs Declarative

- **Imperative (jQuery/vanilla JS):** You tell the computer *how* to do something, step-by-step. "Find this element, then change its text, then add this class."

  Example : You call the pizza place and give them step-by-step instructions: "Preheat your oven, roll the dough, spread the sauce, add cheese, add pepperoni, bake for 15 minutes...

- **Declarative (React):** You tell React *what* you want the UI to look like for a given state, and React figures out the "how" for you. "If the user is logged in, the UI should show a 'Logout' button. If not, show a 'Login' button."

  Example : You just call and say, "I want a large pepperoni pizza." You describe the final state you want and trust the chef (React) to handle the steps to get there.

REACT VS ANGULAR VS VUE

Feature	React	Angular	Vue
Type	Library (focuses on UI)	Framework (full-featured)	Framework (progressive, lightweight)
Language	JavaScript + JSX	TypeScript (mostly)	JavaScript (optionally TypeScript)
Learning Curve	Easy to start, harder for large apps	Steep (lots of concepts)	Moderate, easy to pick up
Data Binding	One-way binding (mostly)	Two-way binding	Two-way binding
DOM	Virtual DOM	Real DOM (but can use change detection)	Virtual DOM

REACT VS ANGULAR VS VUE

Feature	React	Angular	Vue
Community & Popularity	Very big, backed by Facebook	Big, backed by Google	Growing, popular in Asia
Best For	Interactive UI components	Large enterprise apps	Small to medium apps, flexible
Size	Small	Large	Small

Why we use React?

1. Builds Dynamic UIs Easily : React makes it easy to create apps that update in real-time — like liking a post, adding items to a cart, or typing in a search bar — **without reloading the page**.

2. Component-Based Architecture : You can build your UI using **reusable components** — like buttons, cards, or forms — and manage them separately. It's like using Lego blocks to build complex structures.

3. Fast and Efficient : React uses a **Virtual DOM** to update only the parts of the page that change, making your app faster and smoother.

Why we use React?

4. Developer-Friendly : Its syntax (JSX), tools, and community make it easy to **write, debug, and maintain** code — even as your app grows.

5. Huge Ecosystem & Job Demand : React is supported by a massive ecosystem of libraries (like React Router, Redux) and is in **high demand in the job market**.

Check Your Understanding

Question : **What is React?**

Options : a) A complete web framework that includes a server and database.

b) A JavaScript library for building user interfaces.

c) A programming language that compiles to JavaScript.

d) A CSS pre-processor for styling components.

Answer : b) A JavaScript library for building user interfaces.

Explanation: React is specifically a library focused on the "View" layer of an application. It helps developers build interactive and reusable UI components.

References

- **W3Schools – JS Tutorial :** *Beginner-friendly JS tutorials with examples and interactive editors.*
  🔗 https://www.w3schools.com/react

- **react.dev -** *A comprehensive, tutorial-style website that deeply explores core React concepts, including components, state, effects, and hooks, through interactive examples and in-depth explanations.*
  🔗https://react.dev/

Thanks For Watching

K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Course - Web Development - II

welcome@krmangalam.edu.in

Sohna Road,Gurugram,Haryana 122103

Lecture 20:

React Features I

-By Instructor Name

K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Course - Web Development - II

welcome@krmangalam.edu.in

Sohna Road,Gurugram,Haryana 122103

Table of Contents

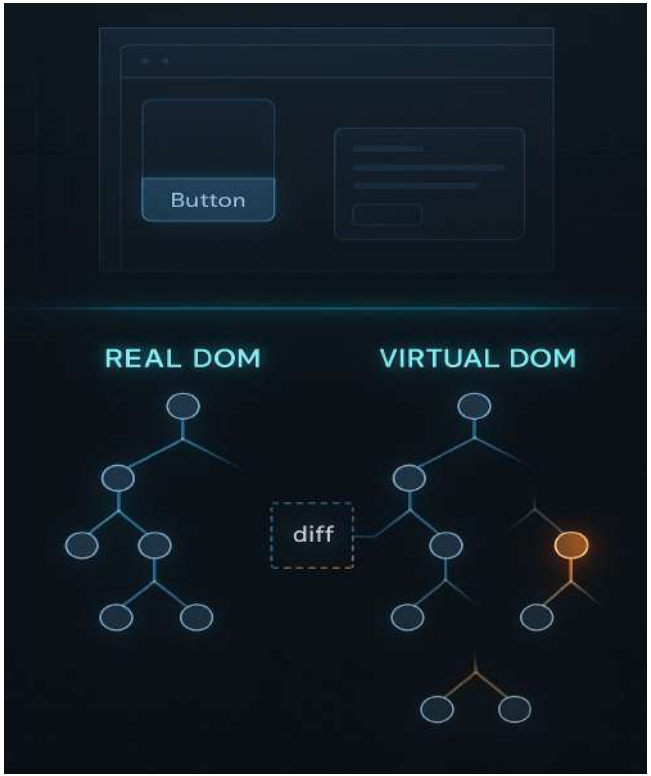
- Virtual DOM vs. Real DOM.
- Reconciliation
- SPA
- JSX Syntax & Expressions.
- References.
- The End.

What would happen if you remodeled(Rebuild) the entire room for every small change?

It would be time-consuming, inefficient, and exhausting — like reloading the entire web page for one small UI change.

React avoids this by using the Virtual DOM.

The **blueprint** represents the **Virtual DOM** — a copy of the real UI used to track and compare changes.



Virtual DOM

The **Virtual DOM (VDOM)** is a **lightweight copy** of the **real DOM** (Document Object Model) used by React to make UI updates faster and more efficient.

React creates a **virtual copy** of the real DOM in memory.

When something changes in the UI (e.g., a user clicks “Like”), React:

- Updates the **Virtual DOM** first.
- Compares the **old Virtual DOM** with the **new Virtual DOM** (called **diffing**).
- Finds what actually changed.

Then, it updates **only that part** of the **real DOM**, not the whole page.

Why is the Virtual DOM important?

- Faster updates
- Better performance
- Smoother user experience
- No full-page reloads

Virtual DOM vs Real DOM

Feature	Real DOM	Virtual DOM
What it is	The actual DOM structure rendered in the browser	A lightweight copy of the Real DOM stored in memory
Update Speed	Slow — updates the whole tree or large parts	Fast — updates only the changed part
Efficiency	Less efficient, causes reflows/repaints	Highly efficient, minimizes direct DOM operations
Created by	Browser	React (in memory)
Update Process	Directly updates the UI	Compares old and new Virtual DOM, then updates Real DOM minimally
Performance	Can slow down large applications	Optimized for better performance in dynamic apps

Reconciliation

- When the state of your application changes, React creates a **new** Virtual DOM tree.
- React then compares this new VDOM with the previous VDOM. This comparison process is called **"diffing."**
- React figures out the absolute minimum number of changes required to make the Real DOM look like the new VDOM.
- It then **batches** these changes and updates the Real DOM in one single, efficient operation.

SPA

An **SPA** is a web app where:

- The **page loads once**
- After that, **only data changes**, not the whole page
- No full-page refresh like traditional websites

Example: **Gmail, Instagram, WhatsApp Web**

A React SPA is a web application that loads a single HTML page and dynamically updates content without reloading the page.

How React makes SPA possible

React updates **only the required part of the UI**, not the whole page.

- Uses **Virtual DOM.**
- Fast UI updates.
- Smooth user experience.
- Uses **BrowserRouter** for client-side routing without page reload.

One downside

- **SEO is harder** (but solvable with SSR / Next.js)

JSX

JSX stands for JavaScript XML. It's a syntax extension for JavaScript.

It is **not** HTML. It gets transpiled by Babel into regular **React.createElement()** function calls.

JSX (JavaScript XML) lets you **write HTML-like code inside JavaScript** — used in React to describe what the UI should look like.

Syntax :

JavaScript

```
1 <h1>Hello, world!</h1>
```

...directly inside your JavaScript code — instead of using document.createElement() or other complex JS methods.

JSX Expressions

Expressions in JSX are snippets of JavaScript that you can **embed inside curly braces {}** to make your UI dynamic.

Example :

JavaScript

```
const name = "John";
<h1>Hello, {name}!</h1>  // JSX Expression → Output: Hello, John!
```

You can use:

- Variables: {name}
- Math: {2 + 2}
- Functions: {getUserName()}
- Conditionals (ternary): {isLoggedIn ? "Logout" : "Login"}

JSX Syntax Rules

Feature	Example
HTML + JS mix	<h1>Hello {name}</h1>
Curly braces for JS	Total: {price * quantity}
One root element	Must return **one parent tag**
Use className	...
Use htmlFor	<label htmlFor="email">Email</label>

Check Your Understanding

Question : **What is JSX?**

Options : a) A different name for JavaScript.

b) A templating language that is completely separate from JavaScript.

c) A syntax extension for JavaScript that looks like HTML.

d) A method for querying the DOM.

Answer : c) A syntax extension for JavaScript that looks like HTML.

Explanation: JSX allows you to write HTML-like syntax directly within your JavaScript code, making it easier to visualize and create the structure of your UI components.

Check Your Understanding

Question : **What is the primary advantage of the Virtual DOM over the Real DOM?**

Options : a) It is more secure.

b) It can be styled directly with CSS.

c) It is a lightweight copy, and updating it is much faster than updating the Real DOM directly.

d) It works without JavaScript.

Answer : c) It is a lightweight copy, and updating it is much faster than updating the Real DOM directly.

Explanation: Manipulating the Real DOM is slow. React creates a Virtual DOM, calculates the most efficient way to apply changes (a process called "diffing"), and then updates the Real DOM only with the necessary changes, leading to better performance.

Check Your Understanding

Question : **How do you embed a JavaScript variable, say userName, inside a JSX element?**

Options : a) <h1>(userName)</h1>

b) <h1>${userName}</h1>

c) <h1>{{userName}}</h1>

d) <h1>{userName}</h1>

Answer : d) <h1>{userName}</h1>

Explanation: Curly braces **{}** are used in JSX to "escape" back into JavaScript. Any valid JavaScript expression can be placed inside them.

References

- **MDN Web Docs :** *Comprehensive and official documentation for JS.*
  🔗https://developer.mozilla.org/en-US/docs/Web/JavaScript

- **W3Schools – JS Tutorial :** *Beginner-friendly JS tutorials with examples and interactive editors.*
  🔗 https://www.w3schools.com/js/

- **react.dev -** *A comprehensive, tutorial-style website that deeply explores core React concepts, including components, state, effects, and hooks, through interactive examples and in-depth explanations.*
  🔗https://react.dev/

Thanks For Watching

K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Course - Web Development - II

welcome@krmangalam.edu.in

Sohna Road,Gurugram,Haryana 122103

Lecture 21:

Vite Setup

-By Instructor Name

K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Course - Web Development - II

welcome@krmangalam.edu.in

Sohna Road,Gurugram,Haryana 122103

Table of Contents

- Introduction to Vite
- Project Creation
- Folder Structure
- References.
- The End.

What is Vite?

Vite (pronounced "veet", French for *fast*) is a **modern build tool** for front-end development that helps you **start and run web projects faster** than older tools like Webpack.

It was created by **Evan You** (the creator of Vue.js) and is now widely used in **React**, **Vue**, **Svelte**, and more.

Think of Vite like a modern microwave:

- Older tools (like Webpack) are like an oven — slow to start and takes time to heat up.
- Vite is instant, fast, and knows **exactly what to heat**, saving time during development.

Why use Vite?

Feature	What It Means
Blazing Fast Startup	Uses **native ES Modules** and skips bundling on dev start
Hot Module Replacement (HMR)	Instantly updates only the changed parts of your app
Zero Config	Works out-of-the-box — minimal setup needed
Optimized Build	Uses **Rollup** under the hood for fast production builds
Framework Agnostic	Works with React, Vue, Svelte, Lit, etc.

Creating First React App with Vite

Step 1: Install Node.js

Make sure Node.js is installed (v14+ recommended).

Check with:

shell

```shell
node -v
npm -v
```

Creating First React App with Vite

Step 2: Create a New Vite + React Project

In your terminal, run:

Terminal

```
1 npm create vite@latest my-react-app
```

Then follow the prompts:

- **Project name:** my-react-app (or your project's custom name)
- **Select framework:** React
- **Select variant:** JavaScript or TypeScript

Creating First React App with Vite

Step 3: Navigate into the Project Folder

Terminal

```
cd my-react-app
```

Step 4: Install Dependencies

Terminal

```
npm install
```

Creating First React App with Vite

Step 5: Start the Development Server

Terminal

```
1 npm run dev
```

Your app will be available at:

http://localhost:5173

Folder Structure

Folder Structure

```
my-react-app/
├── node_modules/
├── public/
│   └── vite.svg
├── src/
│   ├── App.css
│   ├── App.jsx
│   ├── index.css
│   └── main.jsx
├── .gitignore
├── index.html
├── package.json
├── vite.config.js
└── README.md
```

Folder Structure

📂 1. node_modules/

Automatically created when you run npm install.
Contains all the dependencies (packages/libraries) used in your app.

📂 2. public/

Contains **static assets** like images, icons, or fonts that won't change during build.

- vite.svg → Example SVG logo used in the starter project.
- Files here are served directly — e.g., /vite.svg becomes accessible from the root URL.

Folder Structure

📂 **3. src/**

This is where **all your React code** lives. You'll spend most of your time here.

- **App.jsx**
  Main React component — your app's UI starts from here.

- **App.css**
  Styles specific to the App component.

- **main.jsx**
  The **entry point** of the app. It renders <App /> into the HTML page using ReactDOM.createRoot.

- **index.css**
  Global styles that apply to the whole app.

Folder Structure

📄 4. index.html

This is the **single HTML page** where your React app is mounted.
Look for this line:

```
<div id="root"></div>
```

That's where React injects your entire UI.

Folder Structure

📄 **5. package.json**

Contains:

- Project metadata
- Installed dependencies

Scripts like:

```
"scripts": {
  "dev": "vite",
  "build": "vite build",
  "preview": "vite preview"
}
```

Folder Structure

📄 6. vite.config.js

Configuration file for Vite.
You can customize build behavior, define aliases, use plugins, etc.

📄 7. .gitignore

Specifies files/folders to ignore in version control (like node_modules, .env, etc.).

📄 8. README.md

Introductory notes for your project. You can use it to document features, setup instructions, etc.

Check Your Understanding

Question : **What is the main role of a tool like Vite in a React project?**

Options : a) To write the React code for you.

b) To provide a fast development server and an optimized build process for your application.

c) To host the final website online.

d) To check your code for spelling errors.

Answer : d) To provide a fast development server and an optimized build process for your application.

Explanation: Vite provides a modern development environment with features like a super-fast dev server with Hot Module Replacement (HMR) and a pre-configured build setup to bundle your code for production.

References

- **MDN Web Docs :** *Comprehensive and official documentation for JS.*
  🔗https://developer.mozilla.org/en-US/docs/Web/JavaScript

- **W3Schools – JS Tutorial :** *Beginner-friendly JS tutorials with examples and interactive editors.*
  🔗 https://www.w3schools.com/js/

- **react.dev -** *A comprehensive, tutorial-style website that deeply explores core React concepts, including components, state, effects, and hooks, through interactive examples and in-depth explanations.*
  🔗https://react.dev/

Thanks For Watching

K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Course - Web Development - II

welcome@krmangalam.edu.in

Sohna Road,Gurugram,Haryana 122103

Lecture 22:

React Features II

-By Instructor Name

K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Course - Web Development - II

welcome@krmangalam.edu.in

Sohna Road,Gurugram,Haryana 122103

Table of Contents

- One way binding
- Component based (functional, class)
- Default and named export
- References.
- The End.

One-Way Binding

One-way binding in React is a data flow pattern where state is managed in the parent component and passed down to child components through props, and the child cannot directly modify that data.

Parent.jsx

```
1 import React, { useState } from "react";
2 import Child from "./Child";
3
4 function Parent() {
5   const [message, setMessage] = useState("Hello from Parent");
6   return (
7     <div>
8       <h2>Parent Component</h2>
9       <Child text={message} />
10    </div>
11  );
12 }
13 exprt default Parent;
```

One-Way Binding

Child.jsx

```
1 import React from "react";
2
3 function Child(props) {
4   return (
5     <div>
6       <h3>Child Component</h3>
7       <p>{props.text}</p>
8     </div>
9   );
10 }
11
12 export default Child;
```

Why One-Way Binding is Good in React?

- Predictable Data Flow
- Easier Debugging
- Better Control
- Better Performance
- Scalable for Large Applications

React Components

React components are mainly **divided into two formats:**

- **Class Components**
- **Functional Components**.

Class Components

A **Class Component** in React is a component written using an ES6 class. It can store state, use lifecycle methods, and render UI.

A Class Component is a React component created using a JavaScript class that extends React.Component and must contain a **render()** method which returns JSX.

Welcome.jsx

```
import React from "react";

class Welcome extends React.Component {
  render() {
    return <h1>Hello Students</h1>;
  }
}

export default Welcome;
```

Class Components

Explanation

- **class Welcome extends React.Component** → creates a React class component
- **render()** → required method that returns JSX (UI)
- **React.Component** → base class that provides React features like state and lifecycle methods

Key Features of Class Components

- Uses **this.state** to store data
- Uses **this.setState()** to update data
- Supports **lifecycle methods** (componentDidMount, etc.)
- Must include **render()**

Default Export

A **default export** means a file exports **only one main thing**.

You can import it with **any name**.

Student.jsx

```
1 export default function Student() {
2   return <h1>Hello Student</h1>;
3 }
```

App.jsx

```
1 import MyStudent from "./Student";
2
3 function App() {
4   return <MyStudent />;
5 }
```

Default Export

Notice:

- File exported Student
- But we imported as MyStudent
- Name can be anything

Rule:

- Only one **default export** per file
- Imported without curly braces

```
import AnythingName from "./file";
```

Named Export

A **named export** allows you to export **multiple things** from one file.

Name must match while importing.

utils.js

```
1 export function add(a, b) {
2   return a + b;
3 }
4
5 export function subtract(a, b) {
6   return a - b;
7 }
```

App.js

```
1 import { add, subtract } from "./utils";
```

Named Export

Notice:

- Must use **curly braces {}**
- Names must match exactly

Rule:

- You Can Export Multiple Things
- Must Use Curly Braces While Importing
- Import Name Must Match Exactly
- You Can Rename While Importing (Using as)

Default vs Named Export

Feature	Default Export	Named Export
How many per file	Only 1	Multiple allowed
Import syntax	No {}	Use {}
Name while importing	Can change	Must match

Check Your Understanding

Question : **If you have useEffect(() => { ... }, [count]);, when will the effect's callback function be executed?**

Options :

a) Only once, when the component mounts.

b) After every single render of the component.

c) When the component mounts, and then again anytime the **count** state variable changes.

d) Only when the **count** state variable changes, but not on the initial mount.

Answer : c) When the component mounts, and then again anytime the **count** state variable changes.

Explanation: The dependency array tells React to re-run the effect if any of the values inside the array have changed between renders. It always runs on the initial mount as well.

References

- **MDN Web Docs :** *Comprehensive and official documentation for JS.*
  🔗https://developer.mozilla.org/en-US/docs/Web/JavaScript

- **W3Schools – JS Tutorial :** *Beginner-friendly JS tutorials with examples and interactive editors.*
  🔗 https://www.w3schools.com/js/

- **react.dev -** *A comprehensive, tutorial-style website that deeply explores core React concepts, including components, state, effects, and hooks, through interactive examples and in-depth explanations.*
  🔗https://react.dev/

Thanks For Watching

K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Course - Web Development - II

welcome@krmangalam.edu.in

Sohna Road,Gurugram,Haryana 122103

Lecture 23:

States & Props

-By Instructor Name

K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Course - Web Development - II

welcome@krmangalam.edu.in

Sohna Road,Gurugram,Haryana 122103

Table of Contents

- States in Class Components.
- useState Basics.
- Props
- References.
- The End.

State in Class Component

In React **class components**, *state* is an object used to store dynamic data that can change over time and affect what gets rendered.

Key Points:

- State is **managed inside the component**
- When state changes, the component **re-renders automatically**
- It is declared using this.state

State in Class Component

Counter.jsx

```
1 import React, { Component } from "react";
2
3 class Counter extends Component {
4   constructor() {
5     super();
6     this.state = {
7       count: 0
8     };
9   }
10
11  increment = () => {
12    this.setState({ count: this.state.count + 1 });
13  };
14
15  render() {
16    return (
17      <div>
18        <h2>Count: {this.state.count}</h2>
19        <button onClick={this.increment}>Increase</button>
20      </div>
21    );
22  }
23 }
24
25 export default Counter;
```

State in Class Component

Important:

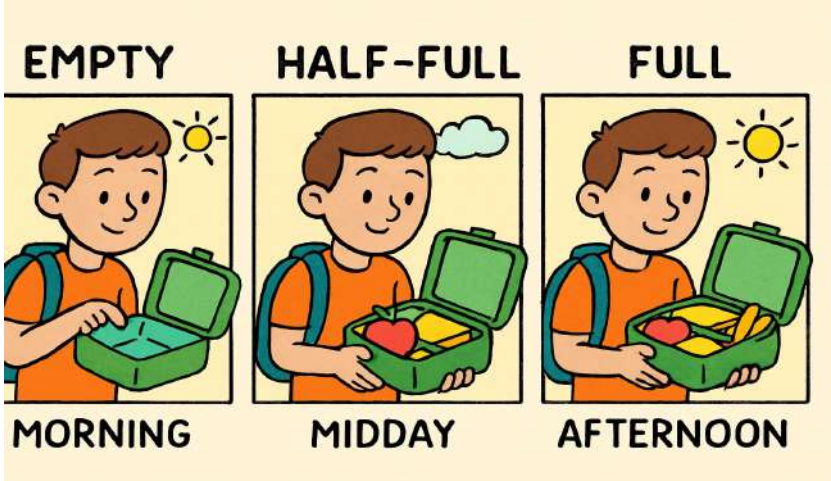
- Use **this.setState()** to update state (not direct assignment)
- State updates are **asynchronous**
- You can store multiple values in state:

Imagine you have a lunchbox that can be empty, half-full, or full. How would you keep track of its current state throughout the day?

Imagine you have a lunchbox that can be empty, half-full, or full. How would you keep track of its current state throughout the day?

To keep track of your lunchbox's state throughout the day, you start by setting an initial state (e.g., empty), update it whenever the lunchbox changes (half-full, full), and use that state to show the current status.

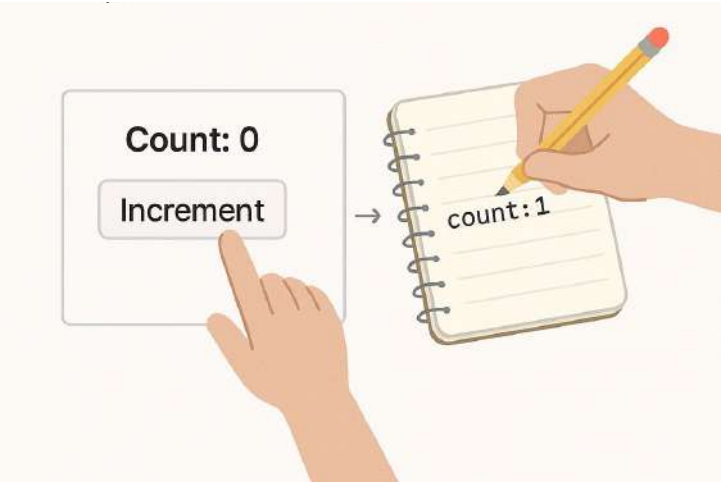
In **React**, this is like using **state** to remember and update dynamic data inside a component.



Introduction to React State

React State is a built-in feature in React that allows components to store and manage dynamic data. It represents values that can change over time due to user interaction, system events, or internal logic.

When state changes, React automatically **re-renders** the component to reflect the new data in the UI.



Introduction to React State

Key Points

- Each React component can have its own state.
- When the state changes, React re-renders the component updating the UI.
- State can only be updated using special functions (not directly).
- You manage state using the **useState()** hook in functional components.

Why do we need React State

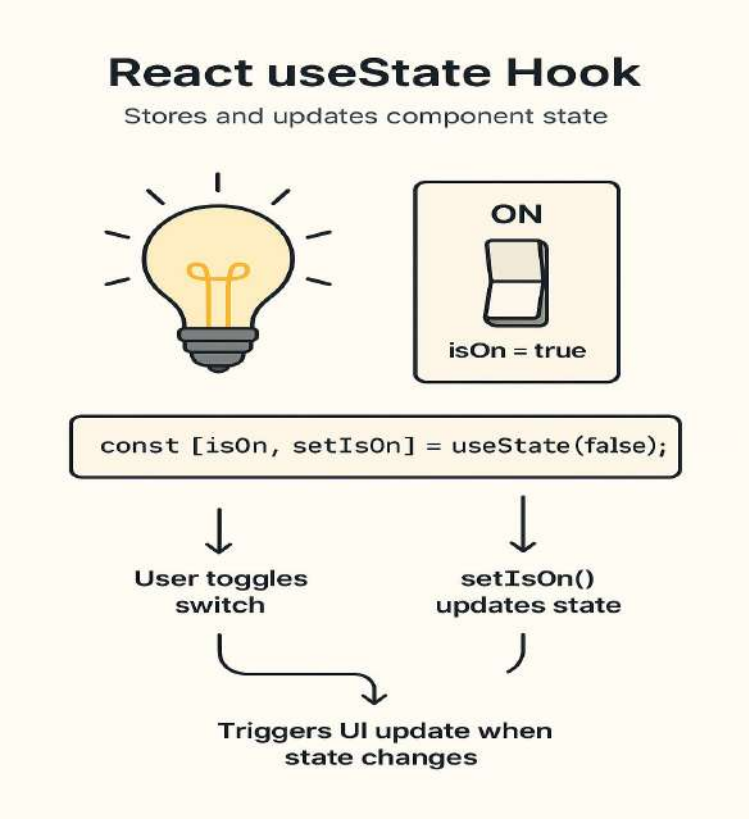
- To **store changing data** in a component.
- To **respond to user actions** (like clicks or typing).
- To **update the UI automatically** when data changes.
- Makes the app **dynamic and interactive**.

Imagine you have a light switch in your room that you can flip on or off. How do you keep track of whether the light is currently on or off?

Imagine you have a light switch in your room that you can flip on or off. How do you keep track of whether the light is currently on or off?

To keep track of whether a light switch is on or off, you store its current status as a true (on) or false (off) value.

In React, the useState() hook does the same by initializing a state variable (like isOn), updating it when toggled, and re-rendering the UI to show the current light status.



useState()

useState() is a **React Hook** that lets you add **state** (a way to store changing data) to your **functional components**.

It lets your component **remember** values — like a counter, a form input, or toggle state.

When the state changes, **React automatically re-renders** the component to reflect the new value in the UI.

Syntax :

```
1 const [value, setValue] = useState(initialValue);
```

useState()

Example :

```
1 const [count, setCount] = useState(0);
```

- count → the current state value
- setCount() → function to update that value
- 0 → the initial value of count

Key Points:

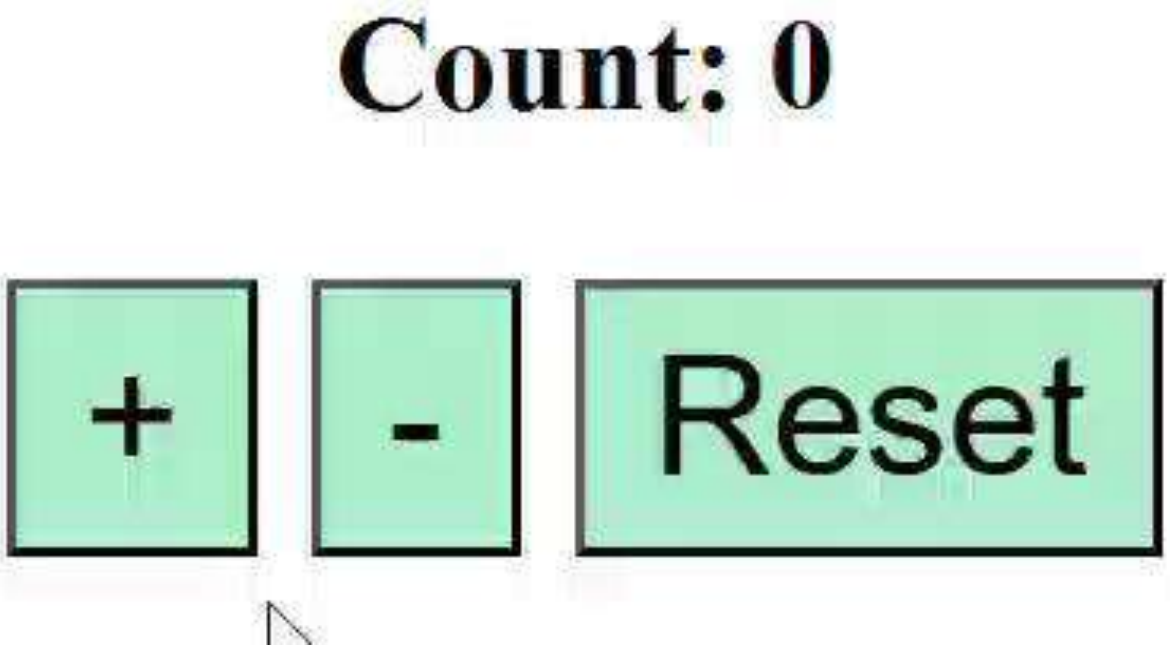
- It comes from React: import { useState } from 'react'
- It can be used **multiple times** to track different values
- You must **not update state directly** — always use the updater function (setCount)

Example

Counter.jsx

```
1 import React, { useState } from 'react';
2
3 function Counter() {
4   const [count, setCount] = useState(0);
5
6   const btn = { padding: '6px 12px', margin: '5px',
7                 fontSize: '30px',backgroundColor:"#adeecbff" };
8
9   const increment = ()=>{
10    setCount(count+1)
11  }
12
13    const decrement = ()=>{
14    setCount(count-1)
15  }
16
17  const reset = ()=>{
18    setCount(0)
19  }
20  return (
21    <div style={{ textAlign: 'center', marginTop: 40 }}>
22      <h2>Count: {count}</h2>
23      <button style={btn} onClick={increment}>+</button>
24      <button style={btn} onClick={decrement}>-</button>
25      <button style={btn} onClick={reset}>Reset</button>
26    </div>
27  );
28 }
29
30 export default Counter;
```

Output



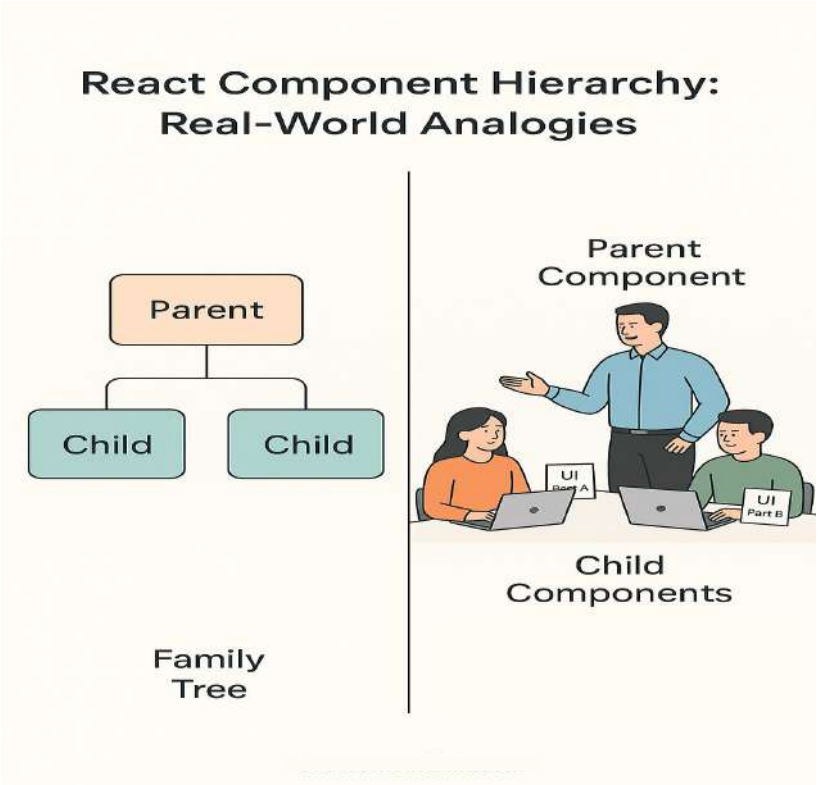
Have you ever seen a family tree or worked on group projects where one person handles a big task and delegates smaller tasks to others?

Have you ever seen a family tree or worked on group projects where one person handles a big task and delegates smaller tasks to others?

Yes! In a family tree, there are parents and children — just like in React, where we have Parent and Child components."

In group projects, there's usually:

- **One team leader (Parent)** who oversees everything and delegates tasks.
- **Team members (Children)** who each handle a specific part of the project.



What if the Parent wants to give the Child some information — like a task description or a tool to use?

What if the Parent wants to give the Child some information — like a task description or a tool to use?

Then the Parent needs a way to pass that information down to the Child — and in React, we do that using props."

Just like in real life:

- A **team leader** might hand a **task sheet** or **toolkit** to a team member.
- In React, the **Parent Component** sends **data (called props)** to the **Child Component** so it can do its specific job.

Think of it like this:

- Task description = prop
- Tool = prop
- Child uses these props to **do its job** but **doesn't change them** — just like a team member follows instructions.

Props

Props (short for **properties**) are a way to **pass data from a parent component to a child component** in React.

Key Points:

- Props are **read-only** — the child **cannot change** them.
- They make components **reusable** by allowing you to **customize** them.
- Passed like **attributes** in HTML.

Example : Props

```
function Parent() {
  return <Child name="Shadav" />;
}

function Child(props) {
  return <h2>Hello, {props.name}!</h2>;
}
```

Program Example : Props

Parent

```
1 import React from 'react';
2 import Button from './Button';
3
4 export default function App() {
5   return (
6     <div style={{ padding: '20px' }}>
7       <h2>Buttons with Props</h2>
8
9       <Button color="blue" text="Submit" />
10      <Button color="green" text="Confirm" />
11      <Button color="red" text="Delete" />
12    </div>
13  );
14 }
```

Child

```
1 import React from 'react';
2
3 export default function Button({ color, text }) {
4   return (
5     <button
6       style={{
7         backgroundColor: color,
8         color: 'white',
9         padding: '10px 20px',
10        border: 'none',
11        borderRadius: '5px',
12        margin: '5px',
13        cursor: 'pointer',
14      }}
15    >
16      {text}
17    </button>
18  );
19 }
```

Output : Props

Coding Questions

1. Passing a String Prop

Task :

1. Create a new component named **Greeting** in a file like **src/Greeting.jsx**.
2. This component should accept a **name** prop.
3. It should render an **<h1>** element that says "Hello, **[name]**!", using the prop.
4. In **App.jsx**, import and use the **Greeting** component, passing your name as the **name** prop

Coding Questions

Solution :

```jsx
// src/Greeting.jsx:

import React from 'react';

function Greeting(props) {
  return <h1>Hello, {props.name}!</h1>;
}

export default Greeting;

// src/App.jsx:

import React from 'react';
import Greeting from './Greeting'; // Import the component

function App() {
  return (
    <div>
      <Greeting name="Alice" />
      <Greeting name="Bob" />
    </div>
  );
}

export default App;
```

Coding Questions

Output :

Hello, Alice!

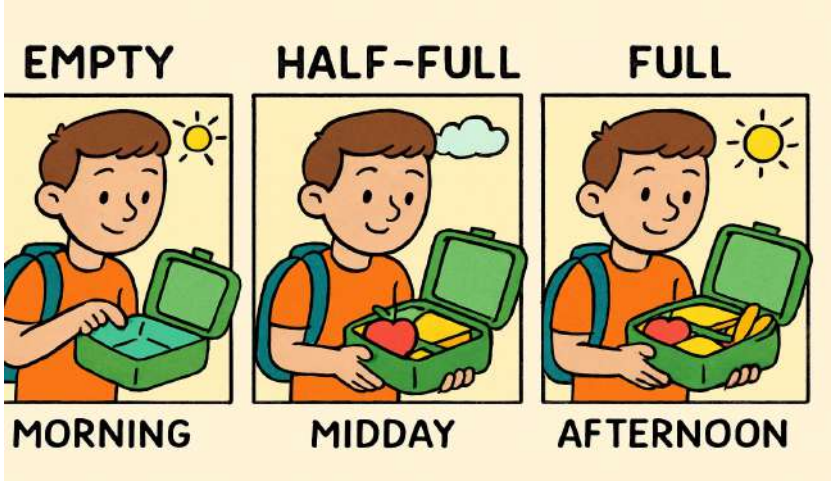
Hello, Bob!

Imagine you have a lunchbox that can be empty, half-full, or full. How would you keep track of its current state throughout the day?

Imagine you have a lunchbox that can be empty, half-full, or full. How would you keep track of its current state throughout the day?

To keep track of your lunchbox's state throughout the day, you start by setting an initial state (e.g., empty), update it whenever the lunchbox changes (half-full, full), and use that state to show the current status.

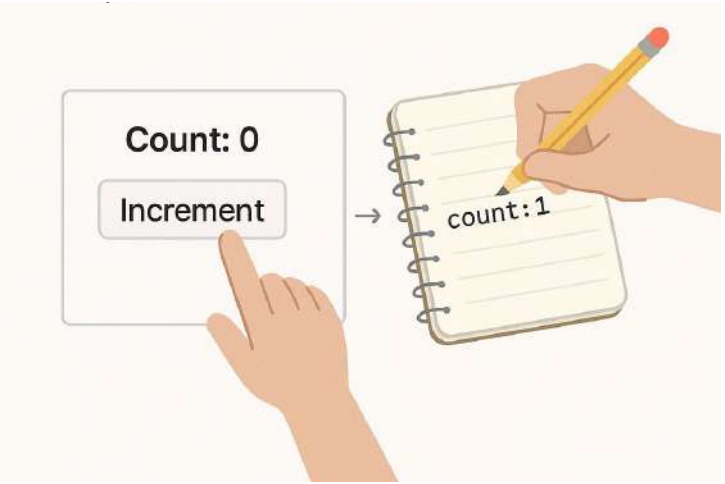
In **React**, this is like using **state** to remember and update dynamic data inside a component.



Introduction to React State

React State is a built-in feature in React that allows components to store and manage dynamic data. It represents values that can change over time due to user interaction, system events, or internal logic.

When state changes, React automatically **re-renders** the component to reflect the new data in the UI.



Introduction to React State

Key Points

- Each React component can have its own state.
- When the state changes, React re-renders the component updating the UI.
- State can only be updated using special functions (not directly).
- You manage state using the **useState()** hook in functional components.

Why do we need React State

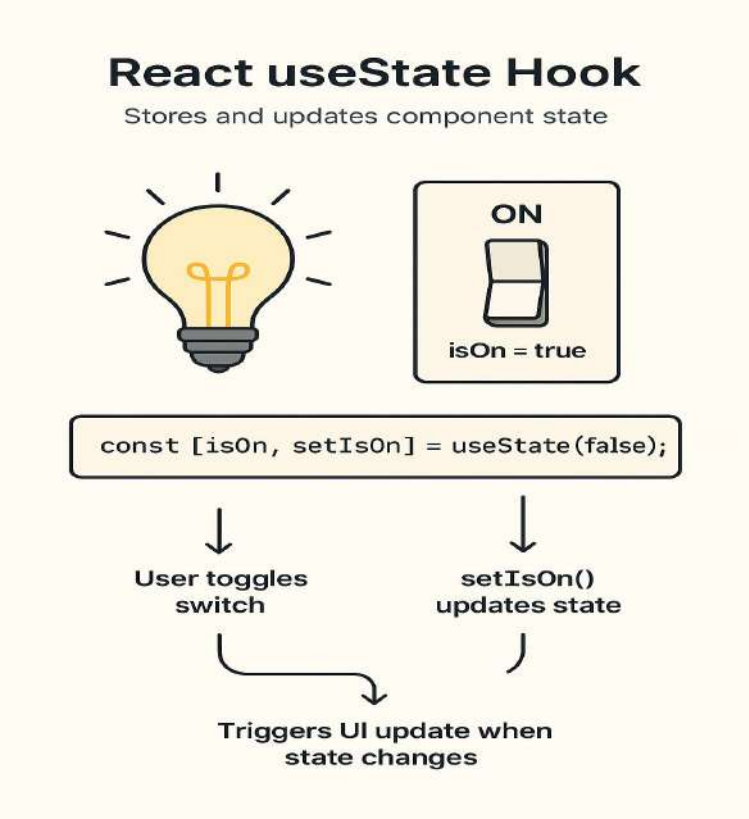
- To **store changing data** in a component.
- To **respond to user actions** (like clicks or typing).
- To **update the UI automatically** when data changes.
- Makes the app **dynamic and interactive**.

Imagine you have a light switch in your room that you can flip on or off. How do you keep track of whether the light is currently on or off?

Imagine you have a light switch in your room that you can flip on or off. How do you keep track of whether the light is currently on or off?

To keep track of whether a light switch is on or off, you store its current status as a true (on) or false (off) value.

In React, the useState() hook does the same by initializing a state variable (like isOn), updating it when toggled, and re-rendering the UI to show the current light status.



useState()

useState() is a **React Hook** that lets you add **state** (a way to store changing data) to your **functional components**.

It lets your component **remember** values — like a counter, a form input, or toggle state.

When the state changes, **React automatically re-renders** the component to reflect the new value in the UI.

Syntax :

```
1 const [value, setValue] = useState(initialValue);
```

useState()

Example :

```
1 const [count, setCount] = useState(0);
```

- count → the current state value
- setCount() → function to update that value
- 0 → the initial value of count

Key Points:

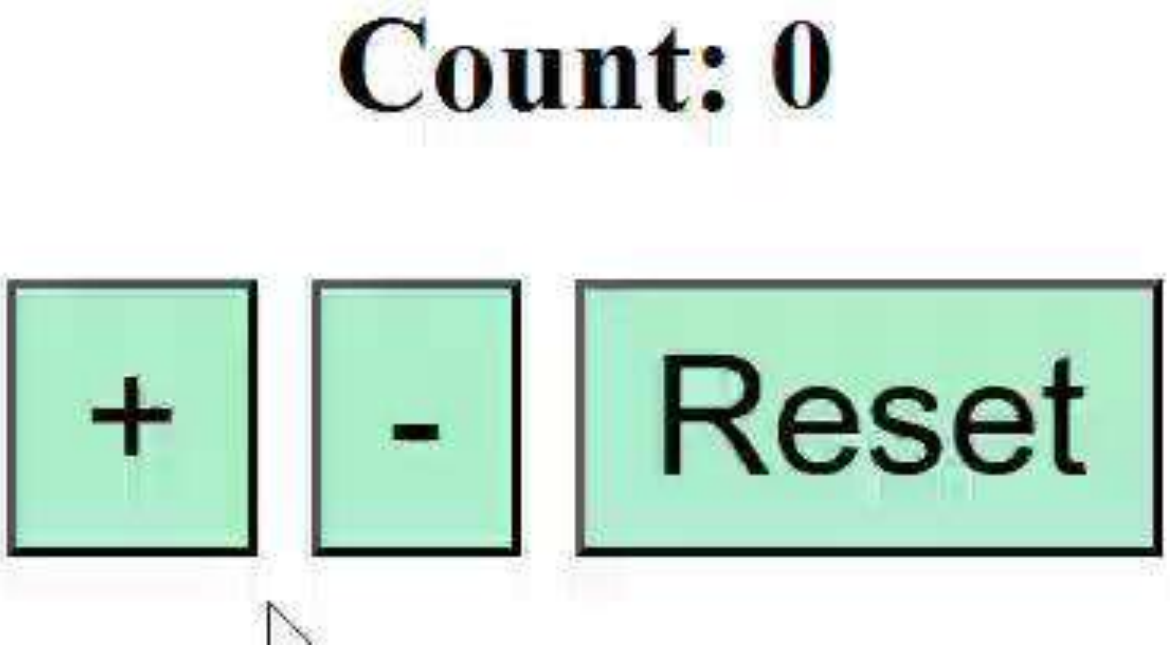
- It comes from React: import { useState } from 'react'
- It can be used **multiple times** to track different values
- You must **not update state directly** — always use the updater function (setCount)

Example

Counter.jsx

```
1 import React, { useState } from 'react';
2
3 function Counter() {
4   const [count, setCount] = useState(0);
5
6   const btn = { padding: '6px 12px', margin: '5px',
7                 fontSize: '30px',backgroundColor:"#adeecbff" };
8
9   const increment = ()=>{
10    setCount(count+1)
11  }
12
13    const decrement = ()=>{
14    setCount(count-1)
15  }
16
17  const reset = ()=>{
18    setCount(0)
19  }
20  return (
21    <div style={{ textAlign: 'center', marginTop: 40 }}>
22      <h2>Count: {count}</h2>
23      <button style={btn} onClick={increment}>+</button>
24      <button style={btn} onClick={decrement}>-</button>
25      <button style={btn} onClick={reset}>Reset</button>
26    </div>
27  );
28 }
29
30 export default Counter;
```

Output



References

- **MDN Web Docs :** *Comprehensive and official documentation for JS.*
  🔗https://developer.mozilla.org/en-US/docs/Web/JavaScript

- **W3Schools – JS Tutorial :** *Beginner-friendly JS tutorials with examples and interactive editors.*
  🔗 https://www.w3schools.com/js/

- **react.dev -** *A comprehensive, tutorial-style website that deeply explores core React concepts, including components, state, effects, and hooks, through interactive examples and in-depth explanations.*
  🔗https://react.dev/

Thanks For Watching

K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Course - Web Development - II

welcome@krmangalam.edu.in

Sohna Road,Gurugram,Haryana 122103

Lecture 24:

Conditional Rendering

-By Instructor Name

K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Course - Web Development - II

welcome@krmangalam.edu.in

Sohna Road,Gurugram,Haryana 122103

Table of Contents

- If/Else
- Ternary Operator
- References
- The End

Did you know React apps behave like a smart assistant – they show different things depending on the situation?

Did you know React apps behave like a smart assistant – they show different things depending on the situation?

Yes, React apps behave like a smart assistant because they use **Conditional Rendering** to display different UI based on conditions.

For example, when data is being fetched, React shows **“Loading…”**, and once the data is available, it displays the actual content.

This is achieved using JavaScript conditions like **if-else or ternary operator** inside components.

IF / ELSE

if-else is used to make decisions in React components, just like in normal JavaScript.

It checks a condition:

- If the condition is **true** → **one UI is shown**
- If the condition is **false** → **another UI is shown**

Note :

In React, if-else is written **outside JSX**, usually before the return.

IF / ELSE

App.jsx

```
1 function App() {
2   const isLoggedIn = true;
3
4   if (isLoggedIn) {
5     return <h1>Welcome User</h1>;
6   } else {
7     return <h1>Please Login</h1>;
8   }
9 }
```

IF / ELSE

How it works

- React checks the condition (isLoggedIn)
- If **true** → **returns "Welcome User"**
- Else → returns **"Please Login"**

When to use

- When logic is **complex**
- When there are **multiple conditions**
- When code becomes messy in JSX

Ternary Operator

The **ternary operator (? :)** is a short way to write if-else in React.

It checks a condition:

- If **true** → **first value**
- If **false** → **second value**

Note:

Ternary is used **inside JSX**, which makes it very useful in React.

Ternary Operator

syntax

```
condition ? trueValue : falseValue
```

App.jsx

```jsx
function App() {
  const isLoggedIn = false;

  return (
    <div>
      {isLoggedIn ? <h1>Welcome User</h1> : <h1>Please Login</h1>}
    </div>
  );
}
```

Ternary Operator

How it works

- React checks isLoggedIn
- If **true** → **shows "Welcome User"**
- If **false** → **shows "Please Login"**

When to use

- When condition is **simple**
- When you want **short and clean code inside JSX**

Check Your Understanding

Question : **Which is correct way to use condition inside JSX?**

Options :

a) if (isLoggedIn) <h1>Welcome</h1>

b) {if (isLoggedIn) <h1>Welcome</h1>}

c) {isLoggedIn ? <h1>Welcome</h1> : <h1>Login</h1>}

d) {isLoggedIn && else <h1>Welcome</h1>}

Answer : c) {isLoggedIn ? <h1>Welcome</h1> : <h1>Login</h1>}

Explanation: JSX does not support if-else directly, so we use the **ternary operator (? :)** to write conditions inside JSX.

References

- **MDN Web Docs :** *Comprehensive and official documentation for JS.*
  🔗https://developer.mozilla.org/en-US/docs/Web/JavaScript

- **W3Schools – JS Tutorial :** *Beginner-friendly JS tutorials with examples and interactive editors.*
  🔗 https://www.w3schools.com/js/

- **react.dev -** *A comprehensive, tutorial-style website that deeply explores core React concepts, including components, state, effects, and hooks, through interactive examples and in-depth explanations.*
  🔗https://react.dev/

Thanks For Watching

K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Course - Web Development - II

welcome@krmangalam.edu.in

Sohna Road,Gurugram,Haryana 122103

Lecture 25:

Life Cycle Methods

-By Instructor Name

K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Course - Web Development - II

welcome@krmangalam.edu.in

Sohna Road,Gurugram,Haryana 122103

Table of Contents

- Life cycle methods
- Mounting
- Updating
- Unmounting
- References.
- The End.

Life Cycle Methods - Class Components

In **React Class Components**, **Lifecycle Methods** are functions that run at different stages of a component's life.

React lifecycle has **three phases**:

- Mounting (Component is created and added to DOM)
- Updating (When state or props change)
- Unmounting (Component removed from DOM)

Life Cycle Method - Mounting

- **constructor()** → used to initialize state
- **render()** → returns JSX (UI)
- **componentDidMount()** → runs after the component is displayed (used for API calls)

Example.jsx

```jsx
class Example extends React.Component {
  constructor() {
    super();
    this.state = { name: "Student" };
  }

  componentDidMount() {
    console.log("Component Mounted");
  }

  render() {
    return <h1>Hello {this.state.name}</h1>;
  }
}
```

Life Cycle Method - Updating

- **shouldComponentUpdate()** → decides whether to re-render
- **componentDidUpdate()** → runs after the update

Counter.jsx

```
import React from "react";

class Counter extends React.Component {
  constructor() {
    super();
    this.state = { count: 0 };
  }

  shouldComponentUpdate(nextProps, nextState) {
    console.log("Should Component Update Called");
    return true; // allow re-render
  }

  componentDidUpdate() {
    console.log("Component Updated");
  }
```

Counter.jsx

```
  render() {
    return (
      <div>
        <h2>Count: {this.state.count}</h2>
        <button onClick={() => this.setState({ count: this.state.count + 1
  })}>
          Increase
        </button>
      </div>
    );
  }
}

export default Counter;
```

Life Cycle Method - Updating

How it works

1. Click **Increase button**
2. setState() changes the state
3. **shouldComponentUpdate()** runs → decides if component should re-render
4. **render()** runs again
5. **componentDidUpdate()** runs after the UI updates

Life Cycle Method - Unmounting

componentWillUnmount() → used to clear timers, remove events, etc. code example.

Timer.jsx

```
1 import React from "react";
2
3 class Timer extends React.Component {
4   constructor() {
5     super();
6     this.state = { count: 0 };
7   }
8
9   componentDidMount() {
10    this.timer = setInterval(() => {
11      this.setState({ count: this.state.count + 1 });
12    }, 1000);
13  }
14
```

Timer.jsx

```
1   componentWillUnmount() {
2     clearInterval(this.timer);
3     console.log("Timer Cleared");
4   }
5
6   render() {
7     return <h2>Timer: {this.state.count}</h2>;
8   }
9 }
10
11 export default Timer;
```

Life Cycle Method - Unmounting

How it works

1. componentDidMount() → starts a **timer**
2. Timer increases **count every second**
3. When the component is **removed from the DOM**
4. componentWillUnmount() runs → **clears the timer**

Drawbacks of Class Components in React

1. ***Complex Syntax***

   Class components use class, constructor, this, and render(), which makes them harder to read and write.

2. ***Use of this Keyword***

   Developers must use this.state, this.props, and bind methods, which can be confusing.

3. ***More Boilerplate Code***

   They require more code compared to functional components.

4. ***Hard to Reuse Logic***

   Logic sharing between components is difficult compared to **Hooks**.

5. ***Lifecycle Methods Replaced by Hooks***

   In modern React, lifecycle methods like **componentDidMount**, **componentDidUpdate**, and **componentWillUnmount** can be handled using the **useEffect hook** in functional components.

References

- **MDN Web Docs :** *Comprehensive and official documentation for JS.*
  🔗https://developer.mozilla.org/en-US/docs/Web/JavaScript

- **W3Schools – JS Tutorial :** *Beginner-friendly JS tutorials with examples and interactive editors.*
  🔗 https://www.w3schools.com/js/

- **react.dev -** *A comprehensive, tutorial-style website that deeply explores core React concepts, including components, state, effects, and hooks, through interactive examples and in-depth explanations.*
  🔗https://react.dev/

Thanks For Watching

K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Course - Web Development - II

welcome@krmangalam.edu.in

Sohna Road,Gurugram,Haryana 122103

Lecture 26:

Styling in React

-By Instructor Name

K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Course - Web Development - II

welcome@krmangalam.edu.in

Sohna Road,Gurugram,Haryana 122103

Table of Contents

- Inline Styling vs. CSS Modules.
- Styled Components & Emotion Library.
- Tailwind CSS in React.
- Theming & Global Styles.
- References.
- The End.

Now that we know how to build components, how do we style them to look modern, clean, and user-friendly?

Now that we know how to build components, how do we style them to look modern, clean, and user-friendly?

Now that we've built the structure of our components, it's time to **give them a visual identity** — just like decorating a house after it's built.

In React, there are **multiple ways to style components**, each with its own use case and benefits.

- Inline Styles.
- CSS Stylesheets.
- CSS Modules.
- Styled Components.

Inline Styles

Apply styles directly to elements using the style prop.

Great for **quick, one-off changes**.

Example :

```
<button style={{ backgroundColor: 'blue', color: 'white' }}>
        Click Me
</button>
```

CSS Stylesheets

Use standard .css files and className in JSX.

Clean and **separates logic from design**.

Example :

```
// App.css
.button {
  background-color: blue;
  color: white;
}

// App.jsx

<button className="button">Click Me</button>
```

CSS Modules

CSS Modules are a way to **scope your CSS to individual components**, so that styles don't leak out or clash with styles from other components.

Instead of using global .css files where class names might collide, **CSS Modules create locally scoped class names** automatically.

Example :

```
import styles from './Button.module.css';

<button className={styles.primary}>
    Click Me
</button>
```

ClassName

Inline Styles vs CSS Modules

Feature	Inline Styles	CSS Modules
Where?	Inside component (style={{}})	Separate .module.css file
Syntax	JS object (camelCase)	Normal CSS (kebab-case)
Scoped?	Yes (element-level)	Yes (auto-generated class names)
Reusable?	No	Yes
Dynamic Styles	Easy with props/state	Needs logic with clsx
Hover/Focus	Not supported	Supported
Media Queries	Not supported	Supported
Theming	Easy	Needs setup

Coding Questions

1. Inline Styling

Task :

1. Create a React component named **AlertBox**. Inside the component, create a JavaScript object named **alertStyle**. This object should define a **backgroundColor** of **lightcoral**, a **color** of **white**, and **padding** of **'16px'**. Apply this style object to a ** ** that contains the text "This is an alert!".

Coding Questions

Solution :

AlertBox.jsx

```jsx
import React from 'react';

function AlertBox() {
  // 1. Create the style object
  const alertStyle = {
    backgroundColor: 'lightcoral',
    color: 'white',
    padding: '16px',
    borderRadius: '8px',
    margin: '10px'
  };

  // 2. Apply the object to the style attribute
  return (
    <div style={alertStyle}>
      This is an alert!
    </div>
  );
}

export default AlertBox;
```

Coding Questions

Output :

This is an alert!

Coding Questions

2. Basic CSS Modules

Task :

1. Create a CSS file named **Button.module.css**. Inside it, define a class **.primaryButton** with some styles (e.g., background color, padding, border).
2. Create a Button component in **Button.jsx**.
3. Import the styles from **Button.module.css**.
4. Apply the imported class to a **<button>** element.

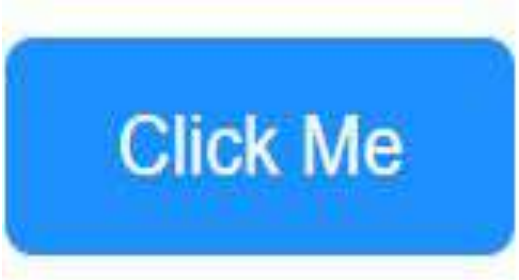
Coding Questions

Solution :

```
//  src/Button.module.css:
// CSS
.primaryButton {
  background-color: dodgerblue;
  color: white;
  padding: 10px 20px;
  border: none;
  border-radius: 5px;
  cursor: pointer;
}

//  src/Button.jsx:
//JS

import React from 'react';
// 1. Import the CSS module file
import styles from './Button.module.css';

function Button() {
  // 2. Apply the class using the imported styles object
  return (
    <button className={styles.primaryButton}>
      Click Me
    </button>
  );
}

export default Button;
```

Coding Questions

Output :



Why switch between CSS and JS files when you could write CSS right where your component lives?

Why switch between CSS and JS files when you could write CSS right where your component lives?

Because writing CSS directly in your component — like with **Styled Components** or **Emotion** — keeps your styles **scoped**, **reusable**, and **easier to maintain**, all in one place.

No more file switching, class name clashes, or scattered styles — just clean, component-driven styling where it belongs.

```
// App.jsx
import styled from
'styled-components';

const Button = styled.button'
   background-color: blue
   color: white;
}
const Card = styled.div'
   padding: 16px
function App() {
   return
     <Card>
      <Button> Click Me<Button>
     </Card>
   }.
}
export default App
```

Styled Components

Uses JavaScript to **create styled components**.

Combines styling + logic in one file — great for **theming and reusable UI**.

Installation : npm install styled-components

Example :

```
import styled from 'styled-components';

const Button = styled.button`
  background-color: blue;
  color: white;
`;

<Button>Click Me</Button>
```

What is Emotion in React ?

Emotion is a popular **CSS-in-JS library** for styling React components.
It lets you write **CSS directly in your JavaScript or TypeScript files**, just like **Styled Components**, but with more flexibility and control.

Think of Emotion as a **styling toolbox** that combines the power of **JavaScript logic** with the ease of **writing regular CSS**.

Installation :

```
npm install @emotion/react @emotion/styled
```

Why use Emotion in React ?

- Style your components **inline or with styled components.**
- **Scoped** styles (no conflicts).
- **Supports themes** (dark mode, custom colors).
- Full **dynamic styling** using props, state, conditions.
- Extremely **fast and lightweight.**

Why use Emotion in React ?

Emotion Has Two Ways to Write Styles:

- **styled API** (like Styled Components).
- css Prop or Function.

Styled API

The **styled API** is a way to create custom, styled HTML or React components using **template literal CSS** — directly in your JavaScript files.

It's used by both:

- styled-components
- @emotion/styled (from Emotion library)

That's why we say:

> **"styled API (like Styled Components)"** — because Emotion and Styled Components both use the same styled syntax and idea.

Styled API

Example :

```
import styled from '@emotion/styled';

const Button = styled.button`
  background-color: teal;
  color: white;
  padding: 10px 20px;
  border-radius: 6px;
  border: none;
  cursor: pointer;
`;

export default function App() {
  return <Button>Click Me</Button>;
}
```

Styled API

Dynamic Styling with Props :

```
1 const Button = styled.button`
2   background-color: ${props => props.primary ? 'blue' : 'gray'};
3   color: white;
4 `;
5
6 <Button primary>Primary Button</Button>
7 <Button>Secondary Button</Button>
```

| css Prop / css Function

The css prop and the css function are two **powerful and flexible** ways to style React components using **Emotion's CSS-in-JS** approach.

These are **more dynamic** and **less tightly coupled** than styled, and are useful when you want:

- Conditional styles
- Inline dynamic styles
- Reusable style blocks without creating new components

Two Ways to Use css in Emotion

1. **css Prop (Directly on Elements)**

- Think of it like applying style={{}}, but with full CSS power — including pseudo-classes, media queries, etc.

Example :

```
/** @jsxImportSource @emotion/react */
import { css } from '@emotion/react';

function App() {
  return (
    <button
      css={{
        backgroundColor: 'hotpink',
        color: 'white',
        padding: '10px 20px',
        borderRadius: '8px',
        '&:hover': {
          backgroundColor: 'deeppink',
        },
      }}
    >
      Click Me
    </button>
  );
}
```

Two Ways to Use css in Emotion

2. css Function (Reusable Style Block)

- Use this when you want to define a CSS block separately and apply it via the css prop.

Example :

```
/** @jsxImportSource @emotion/react */
import { css } from '@emotion/react';

const buttonStyle = css`
  background-color: teal;
  color: white;
  padding: 12px 24px;
  border-radius: 6px;
  border: none;
  &:hover {
    background-color: darkcyan;
  }
`;

function App() {
  return <button css={buttonStyle}>
    Click Me
  </button>;
}
```

CSS Modules separate styles. Styled Components bring them into JS. But what if there's a third way — faster, cleaner, and used by GitHub and Notion?

CSS Modules separate styles. Styled Components bring them into JS. But what if there's a third way — faster, cleaner, and used by GitHub and Notion?

That third way is Utility-First CSS — and the most popular library for it is Tailwind CSS.

With Tailwind, you style your components using **predefined utility classes** directly in your JSX.

No separate CSS files. No styled wrappers. Just fast, consistent, maintainable styling — all in one place.

tailwindcss

Tailwind CSS

Tailwind CSS is a **utility-first CSS framework** that lets you build modern, responsive UIs **directly in your HTML or JSX** using **predefined class names** — no need to write custom CSS.

Key Features:

- **No custom class names** needed
- Perfectly suited for **React** and component-based frameworks
- **Fast development** — you style while you build
- **Predictable and consistent** design system
- Built-in **responsive**, **dark mode**, **hover**, and **animation** utilities

Tailwind CSS Installation & Setup

Step 1 : Installation

terminal

```
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

This creates:

- tailwind.config.js
- postcss.config.js

Tailwind CSS Installation & Setup

Step 2 : Configure Tailwind to Remove Unused CSS

In tailwind.config.js, update the content array:

tailwind.config.js

```
/** @type {import('tailwindcss').Config} */
module.exports = {
  content: [
    "./src/**/*.{js,jsx,ts,tsx}", // important
  ],
  theme: {
    extend: {},
  },
  plugins: [],
}
```

Tailwind CSS Installation & Setup

Step 3 : Add Tailwind Directives to Your CSS

In src/index.css (or App.css), **replace everything** with:

index.css

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

Tailwind CSS Installation & Setup

Step 4 : Use Tailwind Classes in Your JSX

Now you can write Tailwind classes directly in your components:

App.jsx

```
1 export default function App() {
2   return (
3     <div className="text-center p-10 bg-gray-100 min-h-screen">
4       <h1 className="text-4xl font-bold text-blue-600">
5         Welcome to Tailwind CSS
6       </h1>
7       <p className="text-lg mt-4 text-gray-700">
8         Style faster, without writing CSS!
9       </p>
10    </div>
11  );
12 }
```

Tailwind CSS Most Common Classes

Colors

- bg-blue-500 – background color
- text-red-600 – text color
- border-green-400 – border color
- hover:bg-blue-700 – hover background

Spacing (Margin & Padding)

- p-4 – padding on all sides
- px-6 – horizontal padding
- py-2 – vertical padding
- m-4 – margin
- mt-2, mb-2, ml-4, mr-4 – margin top, bottom, left, right

Tailwind CSS Most Common Classes

Typography

- text-lg, text-xl, text-2xl – font sizes
- font-bold, font-semibold, font-light – font weight
- text-center, text-left, text-right – alignment
- uppercase, lowercase, capitalize – text transform

Layout

- flex, flex-row, flex-col – Flexbox layout
- justify-center, justify-between – horizontal alignment
- items-center, items-start – vertical alignment
- grid, grid-cols-2, grid-cols-3 – grid layout
- gap-4, gap-x-2, gap-y-6 – spacing in grids/flex

Tailwind CSS Most Common Classes

Sizing

- w-full, w-1/2, w-64 – width
- h-screen, h-16, h-auto – height
- max-w-md, min-w-full – width constraints

Borders & Radius

- border, border-2 – adds border
- border-gray-300 – sets color
- rounded, rounded-lg, rounded-full – border-radius

Tailwind CSS Most Common Classes

Shadows & Effects

- shadow, shadow-md, shadow-lg – box shadow
- hover:shadow-xl – hover effect
- opacity-75 – transparency

Background & Gradient

- bg-gray-100, bg-white, bg-black – solid backgrounds
- bg-gradient-to-r from-purple-500 to-pink-500 – gradient

Responsive Utilities

- sm:text-sm, md:text-lg, lg:text-xl, xl:text-2xl – responsive font sizes
- hidden md:block – hide on small, show on medium+

Would you rather define your brand’s colors and fonts once, or rewrite them every time you style a button?

Would you rather define your brand’s colors and fonts once, or rewrite them every time you style a button?

Of course, you'd define them once!
Why waste time rewriting colors, fonts, and spacing every time you style a button?

That’s the power of **Theming** in Tailwind CSS — you define your **brand's look** (like fonts, colors, and breakpoints) once in tailwind.config.js, and reuse those styles anywhere with simple utility classes.

Theming in React

Theming means **defining a consistent visual style** (like colors, fonts, spacing, etc.) across your entire app.

In Tailwind, you customize themes using the tailwind.config.js file.

Example :

```
// tailwind.config.js
module.exports = {
  theme: {
    extend: {
      colors: {
        brand: '#1e40af',     // custom primary color
      },
      fontFamily: {
        sans: ['Poppins', 'sans-serif'], // custom font
      },
    },
  },
}
```

Theming in React

Use case :

```
1 <h1 className="text-brand font-sans">Hello</h1>
```

This helps you build a **branded, consistent UI** instead of mixing styles randomly.

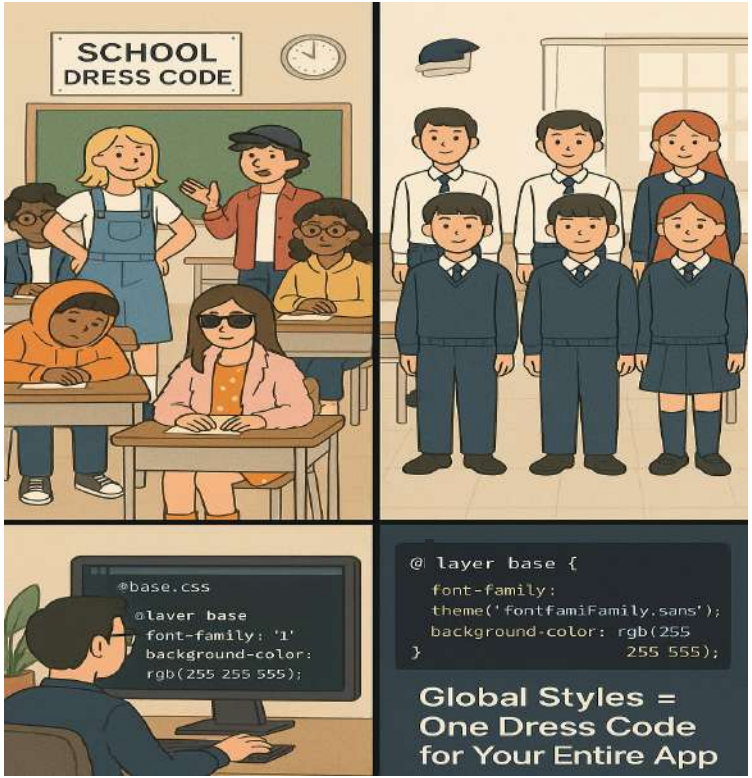
Would you give every student different uniforms — or enforce one dress code for all?

Would you give every student different uniforms — or enforce one dress code for all?

Of course, you'd enforce one dress code for all!
It ensures consistency, clarity, and a sense of unity — no confusion, no clashes.

That's exactly what **Global Styles** do in Tailwind CSS.

They apply **base-level styling** (like default fonts, background colors, heading sizes, etc.) **across your entire app**, so every page and element starts from the same consistent foundation — just like everyone wearing the same dress code, no matter which classroom they're in.



Global Styles - Base Layer Styling

Global styles are default base styles applied across all elements, like resetting margins, body font, or button behavior.

In Tailwind, global styles are handled via:

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

Global Styles - Base Layer Styling

You can also add **custom global styles** inside your main CSS file:

```
@layer base {
  body {
    @apply bg-gray-100 font-sans text-gray-900;
  }

  h1 {
    @apply text-3xl font-bold;
  }
}
```

All <body> elements will use bg-gray-100, font-sans, and text-gray-900 by default,
and all <h1> tags will have text-3xl and font-bold — unless overridden with custom utility classes.

Global Styles vs Theming

Feature	Global Styles	Theming
Purpose	Sets **base/default styles** across your entire app (e.g., fonts, background)	Defines **design tokens** like colors, spacing, and typography for reuse
Scope	Applies universally to all elements unless overridden	Applied through context or config (e.g., theme provider or Tailwind config)
Example Use	Setting a body font or default heading size	Defining a primary color or font family to use consistently across components
Goal	Provide a **consistent base** or reset for the app	Provide a **customizable design system**
How It's Used	Tailwind's @layer base or CSS resets	Tailwind's theme.extend or styled-components' ThemeProvider
Override Style?	You override styles at component level	You reference theme tokens for consistent values

Check Your Understanding

Question : **How do you apply inline styles to a React element?**

Options : a) style="color: 'blue';"

b) style={{ color: 'blue' }}

c) style="color: blue"

d) style={ color: blue }

Answer : b) style={{ color: 'blue' }}

Explanation: Inline styles in React are passed as a JavaScript object. The outer curly braces **{}** enter JSX into JavaScript mode, and the inner curly braces {} define the style object itself.

Check Your Understanding

Question : **What is the main benefit of using CSS Modules?**

Options : a) It allows you to write CSS directly inside your JavaScript files.

b) It automatically scopes your CSS class names locally to the component, preventing naming conflicts.

c) It is the only way to use animations in React.

d) It combines all your CSS into a single global file.

Answer : b) It automatically scopes your CSS class names locally to the component, preventing naming conflicts.

Explanation: CSS Modules solves the global scope problem of traditional CSS by generating unique class names for each component (e.g., **Component_className_1a2b3c**), ensuring styles don't leak out and affect other components.

Check Your Understanding

Question : **What is Tailwind CSS?**

Options : a) A JavaScript library for creating animations.

b) A component library like Material-UI.

c) A utility-first CSS framework for rapidly building custom designs directly in your markup.

d) A library for creating styled-components.

Answer : c) A utility-first CSS framework for rapidly building custom designs directly in your markup.

Explanation: Tailwind provides low-level utility classes (like **p-4, flex, text-lg**) that you compose together in your JSX to build designs without writing custom CSS.

Check Your Understanding

Question : **When using inline styles in React, how would you write the CSS property font-size?**

Options : a) font-size

b) 'font-size'

c) fontSize

d) font.size

Answer : c) fontSize

Explanation: CSS properties that contain a hyphen (kebab-case) must be converted to camelCase when used in JavaScript style objects.

Check Your Understanding

Question : **What is the core concept of CSS-in-JS libraries like Styled Components or Emotion?**

Options : a) They link external **.css** files to your components.

b) They allow you to write actual CSS code within your JavaScript files to create React components with styles attached.

c) They convert all your CSS into inline styles.

d) They only work with class components.

Answer : b) They allow you to write actual CSS code within your JavaScript files to create React components with styles attached.

Explanation: CSS-in-JS libraries let you define components with their styles encapsulated together, often using tagged template literals. This colocates the component's logic and styling.

References

- **MDN Web Docs :** *Comprehensive and official documentation for JS.*
  🔗https://developer.mozilla.org/en-US/docs/Web/JavaScript

- **W3Schools – JS Tutorial :** *Beginner-friendly JS tutorials with examples and interactive editors.*
  🔗 https://www.w3schools.com/js/

- **react.dev -** *A comprehensive, tutorial-style website that deeply explores core React concepts, including components, state, effects, and hooks, through interactive examples and in-depth explanations.*
  🔗https://react.dev/

Thanks For Watching

K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Course - Web Development - II

welcome@krmangalam.edu.in

Sohna Road,Gurugram,Haryana 122103